

NOM Prénom

Programmation Ingénierie des Médias 1 (ProgIM1)**Examen d'entraînement - Partie 1**

24.01.2026

Durée : 90 minutes

	Points obtenus	Points totaux
Question 1		6
Question 2		6
Question 3		4
Question 4		6
Question 5		7
Question 6		6
Question 7		6
Question 8		5
Question 9		4
Total		50
Note finale		

Vous n'avez pas le droit à des pages de résumé pour cette partie. Vous pouvez répondre aux questions en français ou en anglais. Toute tentative de triche sera sanctionnée par la note de 1.

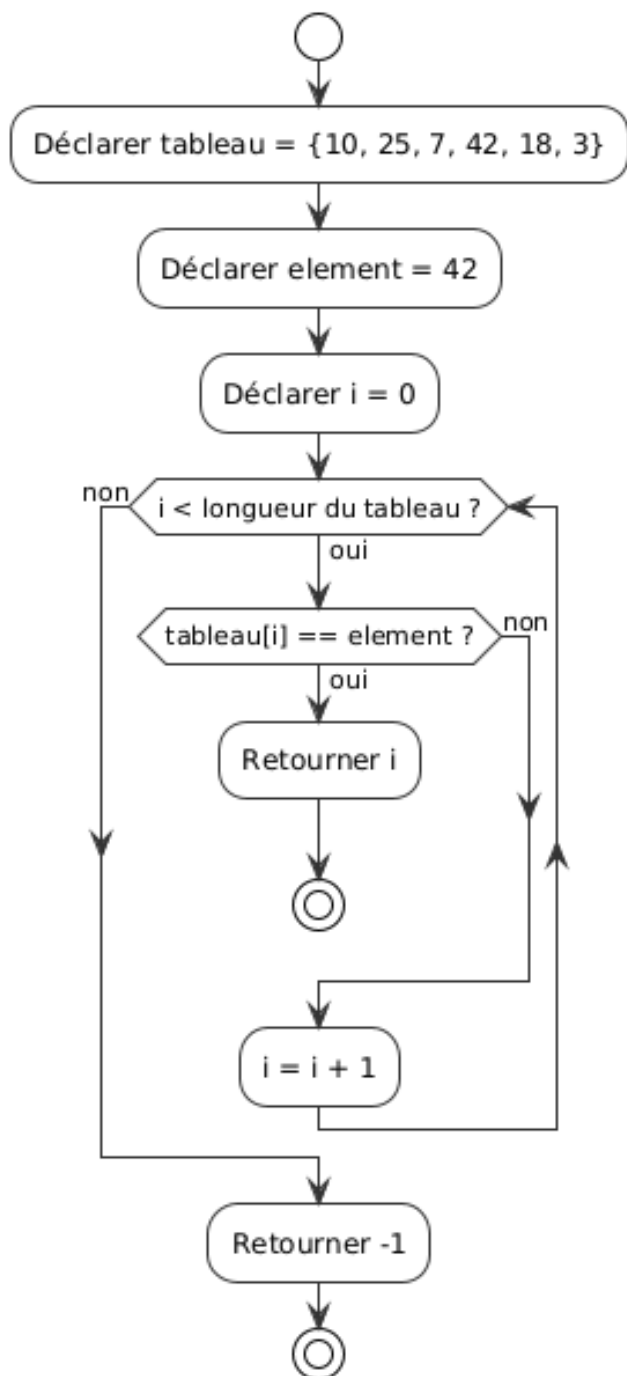
Examen d'entraînement préparé avec l'aide de Ludovic Delafontaine, Hadrien Louis, GitHub Copilot et Claude (Anthropic).

Page laissée vide

1. Lecture de diagramme d'activité

/ 6 pts

Le diagramme d'activité UML ci-dessous représente un algorithme.



Partie A (3 points) : Expliquez en quelques phrases ce que fait cet algorithme.

Partie B (3 points) : Créez votre propre diagramme d'activité UML pour un algorithme qui calcule la somme de tous les éléments d'un tableau d'entiers.

Partie A : Explication de l'algorithme	Points
Explication (3 points) : Cet algorithme recherche un élément (42) dans un tableau d'entiers. Il parcourt le tableau élément par élément avec une boucle. Si l'élément est trouvé, il retourne sa position (index). Si l'élément n'est pas trouvé après avoir parcouru tout le tableau, il retourne -1. (1 point pour identifier qu'il s'agit d'une recherche, 1 point pour le parcours, 1 point pour les valeurs de retour)	
Partie B : Diagramme d'activité pour calcul de somme	Points
Éléments attendus (3 points) : • Début/Fin (0.5 point) • Déclaration du tableau et d'une variable somme = 0 (0.5 point) • Boucle parcourant tous les éléments du tableau (1 point) • Addition de chaque élément à la somme (0.5 point) • Retour ou affichage de la somme (0.5 point) Structure attendue approximative : • Début • somme = 0, i = 0 • Tant que i < longueur tableau ▸ somme = somme + tableau[i] ▸ i = i + 1 • Retourner somme • Fin	

2. Vocabulaire et concepts Java

/ 6 pts

Pour chacune des affirmations suivantes, indiquez si elle est **vraie** ou **fausse**.

Affirmation	Vrai/Faux
En Java, un paramètre est la valeur concrète passée lors de l'appel d'une méthode, tandis qu'un argument est la variable déclarée dans la signature de la méthode.	Faux (c'est l'inverse)
Le JDK (Java Development Kit) contient le compilateur Java ainsi que la JVM (Java Virtual Machine).	Vrai
Un fichier .java est compilé en fichier .class qui contient du bytecode exécutable par la JVM.	Vrai
En Java, une variable déclarée avec le mot-clé 'final' ne peut plus être modifiée après son initialisation.	Vrai
La compilation transforme le code source en langage machine directement exécutable par le processeur.	Faux (elle produit du bytecode pour la JVM)
Une méthode et une fonction sont exactement la même chose en programmation.	Faux (une méthode est une fonction dans une classe)

3. Types primitifs en Java

/ 4 pts

1. Donnez 4 types primitifs en Java parmi les 8 existants. **(2 points)**
2. Pourquoi dit-on que les nombres à virgule flottante (float, double) ne sont pas précis pour représenter certaines valeurs décimales ? Donnez un exemple. **(2 points)**

1. Types primitifs (2 points) : • int, double, boolean, char • Ou : byte, short, long, float (0.5 point par type correct) 2. Imprécision des flottants (2 points) : Les nombres à virgule flottante utilisent une représentation binaire qui ne peut pas représenter exactement certaines fractions décimales. Exemple : $0.1 + 0.2$ donne 0.30000000000000004 et non 0.3 exactement. (1 point pour l'explication, 1 point pour l'exemple)	Points
---	--------

4. Portée des variables

/ 6 pts

Le code Java ci-dessous contient **exactement 3 erreurs** liées à la portée des variables.

Pour chaque erreur :

1. Indiquez la/les ligne(s) concernée(s)
2. Expliquez ce qui ne fonctionne pas et pourquoi

```
public class PorteeVariables {
    public static int compteur = 0;

    public static void main(String[] args) {
        int nombre = 10;

        if (nombre > 5) {
            int resultat = nombre * 2;
            compteur = compteur + 1;
        }

        System.out.println("Résultat : " + resultat);

        for (int i = 0; i < 3; i++) {
            int valeur = i * 2;
        }

        System.out.println("Valeur finale : " + valeur);

        calculer();
    }

    public static void calculer() {
        int somme = nombre + 5;
        System.out.println("Somme : " + somme);
    }
}
```

Les 3 erreurs (6 points, 2 points par erreur identifiée et expliquée) :	Points
1. Ligne 12 : La variable <code>resultat</code> est déclarée dans le bloc <code>if</code> et n'est pas accessible en dehors de ce bloc. Elle n'existe plus après la ligne 10. 2. Ligne 18 : La variable <code>valeur</code> est déclarée dans la boucle <code>for</code> et n'est accessible que dans cette boucle. Elle n'existe pas en dehors. 3. Ligne 23 : La variable <code>nombre</code> est locale à la méthode <code>main</code> et n'est pas accessible dans la méthode <code>calculer</code>. (Note : Il n'y a en fait que 3 erreurs principales. Si l'étudiant-e en trouve d'autres logiques, les accepter)	

5. Bonnes pratiques de programmation

/ 6 pts

Voici un extrait de code Java. Identifiez les problèmes liés aux **conventions de nommage Java** vues en cours et proposez des corrections.

Partie A (3 points) : Conventions de nommage

```
public class CalculateurStatistiques {  
    public static final double taux_tva = 0.077;  
    private int NombreEtudiants;  
  
    public void CalculerMoyenne(int[] notes) {  
        double m = 0.0;  
        // ...  
    }  
  
    private String Nom_Complet;  
}
```

Pour chaque problème identifié, indiquez :

- La ligne concernée
- Le problème (quelle convention Java n'est pas respectée)
- La correction

Partie B (3 points) : Pour chacun des commentaires suivants, indiquez s'il est **utile** ou **redondant** et justifiez brièvement :

```
// Incrémente le compteur de 1  
compteur++;  
  
// Vérifie que l'âge est valide pour voter (18+)  
if (age >= 18) {  
    peutVoter = true;  
}  
  
// Calcule le prix TTC en ajoutant la TVA de 20%  
double prixTTC = prixHT * 1.20;
```

Partie A (3 points) :	Points
1. Ligne 2 : <code>taux_tva</code> → <code>TAUX_TVA</code> • Problème : Les constantes (<code>final static</code>) doivent être en MAJUSCULES_AVEC_UNDERSCORE • (0.6 point)	
2. Ligne 3 : <code>NombreEtudiants</code> → <code>nombreEtudiants</code> • Problème : Les attributs (variables d'instance) doivent commencer par une minuscule (<code>camelCase</code>) • (0.6 point)	
3. Ligne 5 : <code>CalculerMoyenne</code> → <code>calculerMoyenne</code> • Problème : Les méthodes doivent commencer par une minuscule (<code>camelCase</code>) • (0.6 point)	
4. Ligne 6 : <code>m</code> → <code>moyenne</code> ou <code>somme</code> (nom plus explicite) • Problème : Les noms de variables doivent être explicites (convention vue en cours) • (0.6 point)	
5. Ligne 10 : <code>Nom_Complet</code> → <code>nomComplet</code> • Problème : Les attributs doivent être en <code>camelCase</code>, pas de underscore ni majuscule au début • (0.6 point)	
Partie B (3 points) : 1. Redondant : Le commentaire ne fait que répéter ce que le code fait de manière évidente. (1 point) 2. Utile : Le commentaire explique la logique métier (âge pour voter), pas juste ce que fait le code. (1 point) 3. Moyennement utile : Explique le taux de TVA ce qui peut être utile, mais pourrait être mieux avec une constante <code>TAUX_TVA</code>. (1 point)	

6. Packages et bibliothèques Java

/ 6 pts

1. À quoi sert un package en Java ? (2 points)
2. Quelle est la différence entre `import java.util.*;` et `import java.util.Scanner;` ? Quelle forme est préférable et pourquoi ? (3 points)
3. Indiquez brièvement l'utilité de ces trois packages de la bibliothèque standard Java : (1.5 points)
 - `java.util`
 - `java.io`
 - `java.time`

1. Utilité des packages (2 points) :	Points
Un package permet d'organiser les classes Java de manière logique et hiérarchique, éviter les conflits de noms, et faciliter la réutilisation du code.	
2. Différence import (2.5 points) : • <code>import java.util.*;</code> importe toutes les classes du package <code>java.util</code> • <code>import java.util.Scanner;</code> importe uniquement la classe <code>Scanner</code> La forme spécifique (<code>Scanner</code>) est préférable car : • Plus claire : on voit exactement quelles classes sont utilisées • Évite les conflits de noms • Meilleure performance de compilation 3. Utilité des packages (1.5 points - 0.5 point par package) : • <code>java.util</code> : classes utilitaires (collections, dates, <code>Scanner</code>, etc.) • <code>java.io</code> : entrées/sorties (lecture/écriture de fichiers) • <code>java.time</code> : gestion moderne des dates et heures	

7. Compilation et exécution Java en ligne de commande

/ 5 pts

Vous avez un fichier `Calculatrice.java` qui contient une classe publique `Calculatrice` avec une méthode `main` qui accepte deux arguments (deux nombres à additionner).

1. Donnez la commande pour compiler ce fichier. **(1.5 points)**
2. Donnez la commande pour exécuter le programme avec les arguments 10 et 25. **(1.5 points)**
3. Expliquez ce qui se passe lors de la compilation d'un fichier Java. **(2 points)**

Question 1 : Compilation	Points
<code>javac Calculatrice.java</code> (1.5 point pour la commande correcte)	

Question 2 : Exécution	Points
<code>java Calculatrice 10 25</code> (1.5 point pour la commande correcte, noter l'absence de .class)	

Question 3 : Explication	Points
Processus de compilation (2 points) : 1. Le compilateur Java (<code>javac</code>) lit le fichier source <code>.java</code> 2. Il vérifie la syntaxe et la cohérence du code 3. Il traduit le code Java en bytecode (instructions pour la JVM) 4. Il génère un fichier <code>.class</code> contenant ce bytecode 5. Ce bytecode peut ensuite être exécuté par la JVM sur n'importe quelle plateforme (2 points pour une explication complète du processus)	

8. Identification d'erreurs de syntaxe

/ 5 pts

Le code Java ci-dessous contient **exactement 5 erreurs de syntaxe** qui empêcheront la compilation.

Pour chaque erreur :

1. Indiquez la/les ligne(s) concernée(s)
2. Expliquez quelle règle de syntaxe Java n'est pas respectée
3. Proposez la correction

```
public class CalculMoyenne {  
    public static void main(String[] args) {  
        int[] notes = {5, 4, 6, 3, 5}  
        double somme = 0;  
  
        for (int i = 0, i < notes.length, i++) {  
            somme = somme + notes[i];  
        }  
  
        double moyenne = somme / notes.length;  
  
        System.out.println("La moyenne est : " + moyenne)  
  
        if (moyenne >= 4) {  
            System.out.println("Réussi !");  
        }  
    }  
}
```

Les 5 erreurs de syntaxe (1 point par erreur identifiée et corrigée) :	Points
1. Ligne 3 : Il manque un point-virgule ; à la fin de la déclaration du tableau • Correction : <code>int[] notes = {5, 4, 6, 3, 5};</code> 2. Ligne 6 : Dans la boucle for, la première virgule devrait être un point-virgule • Correction : <code>for (int i = 0; i < notes.length, i++)</code> 3. Ligne 6 : Dans la boucle for, la deuxième virgule devrait être un point-virgule • Correction : <code>for (int i = 0; i < notes.length; i++)</code> 4. Ligne 12 : Il manque un point-virgule ; à la fin du <code>println</code> • Correction : <code>System.out.println("La moyenne est : " + moyenne);</code> 5. Ligne 14 : Il manque une accolade ouvrante { après la condition if • Note : Le code compile sans accolades pour un if avec une seule instruction, mais c'est une mauvaise pratique • Correction : <code>if (moyenne >= 4) {</code> Code corrigé complet : <pre>public class CalculMoyenne { public static void main(String[] args) { int[] notes = {5, 4, 6, 3, 5}; double somme = 0; for (int i = 0; i < notes.length; i++) { somme = somme + notes[i]; } double moyenne = somme / notes.length; System.out.println("La moyenne est : " + moyenne); if (moyenne >= 4) { System.out.println("Réussi !"); } } }</pre>	

9. Question 9

10. Analyse de bonnes pratiques

/ 4 pts

Voici deux versions d'un code Java qui affiche un message selon la valeur d'une moyenne :

Version A :

```
if (moyenne >= 4)
    System.out.println("Réussi !");
else
    System.out.println("Échoué !");
```

Version B :

```
if (moyenne >= 4) {
    System.out.println("Réussi !");
} else {
    System.out.println("Échoué !");
}
```

Répondez aux questions suivantes :

1. La version A compile-t-elle sans erreur ? Justifiez brièvement.
2. La version B compile-t-elle sans erreur ? Justifiez brièvement.
3. Laquelle des deux versions recommanderiez-vous d'utiliser ? Expliquez votre choix en mentionnant au moins deux raisons liées aux bonnes pratiques de programmation.

Réponses attendues :	Points
1. La version A compile-t-elle ? (1 point) • Oui, la version A compile sans erreur. • En Java, quand il n'y a qu'une seule instruction après if ou else, les accolades sont optionnelles. • Le compilateur accepte cette syntaxe. 2. La version B compile-t-elle ? (1 point) • Oui, la version B compile sans erreur. • Les accolades {} sont toujours acceptées pour délimiter les blocs d'instructions. 3. Quelle version recommander ? (2 points - 1 point pour le choix, 1 point pour 2 raisons) • Recommandation : Version B (avec accolades) • Raisons possibles : ▸ Lisibilité : Les accolades rendent la structure du code plus claire et explicite ▸ Prévention d'erreurs : Si on ajoute plus tard une deuxième instruction, on pourrait oublier d'ajouter les accolades, créant un bug silencieux ▸ Cohérence : Le code reste uniforme même avec une seule instruction ▸ Maintenabilité : Plus facile à modifier et moins risqué lors de modifications futures ▸ Standards de l'industrie : La plupart des guides de style Java (Google, Oracle) recommandent toujours utiliser les accolades (Accepter la version B avec au moins 2 raisons valables parmi celles-ci ou équivalentes)	